C-KAR: Application of LLM World Knowledge on Career Sequence Datasets

Research Progress Report

Lee Ze Hao

Abstract

Career sequence prediction is a vital topic in human capital management. Prediction of the next job from an individual's career sequence can be formulated as a recommendation problem, where the target is the next position and/or company. Traditional recommendation models are limited to the information contained in its training dataset only. This particularly harms cases with sparse training data, such as small companies with few occurrences in the dataset, or out-of-distribution (OOD) data, such as companies that are newly added and have not appeared before in the training dataset. It has been found that integrating external world knowledge from frozen pre-trained large language models (LLMs) helps bridge the knowledge gap without requiring a larger training dataset. We obtain the Career-KAR (C-KAR) model by creating residual knowledge adaptation experts, and introducing new features to the KAR model from Xi et. al. [1]. We apply C-KAR to next-job-position and next-job-company prediction tasks, finding improved results compared to both KAR and baseline recommendation models.

1 Introduction

In current human capital applications, machine learning models are often used to predict an individual's future jobs. The prediction problem can be formulated as a recommendation system problem, with the sequence of previous jobs as the input, and the following job as target predictions. Job recommendation and matching is an important topic for individual career planning, human resources management, and job platform recommendations.

Traditionally, prediction models are trained only using career-related datasets, such as the results of surveys, or resume datastores. The prediction model's final performance is limited by the size of the dataset, and by the items that appear in the dataset. As a result, these traditional models perform poorly in two aspects: 1. OOD (out-of-distribution) data inputs, and 2. generalization from limited data. These are crucial as novel data (such as new companies) is often introduced in live conditions. This paper proposes that models augmented with LLM world knowledge can address these problems.

In recent years, LLMs pre-trained on large corpuses have proven to achieve good reasoning and knowledge performance. By adapting this knowledge to use in a smaller recommendation backbone model, we can make use of this information without requiring additional training on a large dataset. In this paper, we extend the KAR model proposed by Xi et. al. [1], which adapts outputs from LLMs as useful features in recommendation models, with certain improvements, and apply it to our career sequence dataset. The results prove that by incorporating reasoning and world knowledge from frozen pretrained models, good performance can be achieved even using simple backbone recommendation models with a small amount of trainable parameters, on a dataset of limited size.

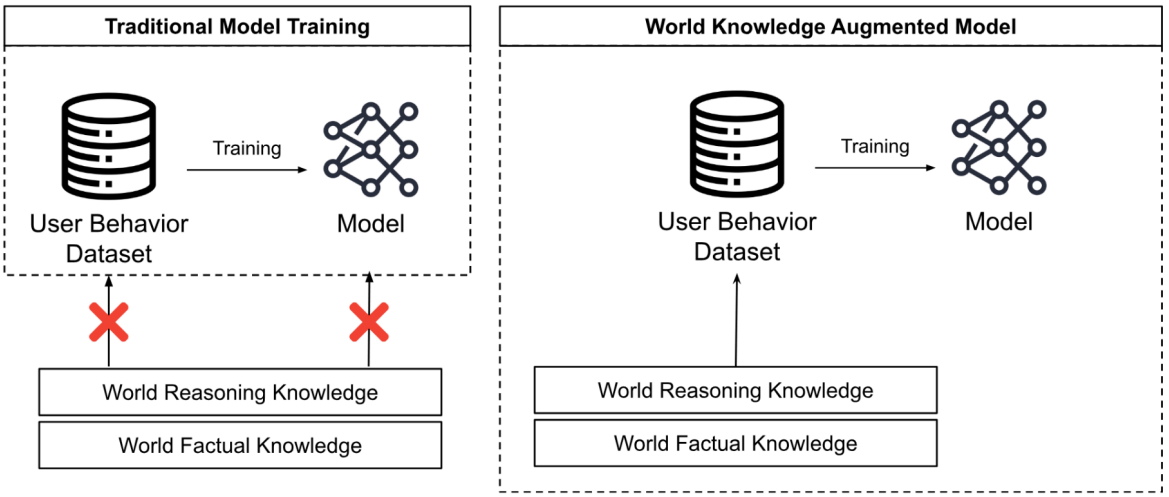


Figure 1: Comparison between information flow of traditional prediction networks and world knowledge augmented models.

2 Related Work

This section reviews prior related applications of LLMs in recommender systems.

2.1 Recommender Systems

Recommender systems are commonly used to provide content recommendations (e.g. products, advertisements, videos) for online applications. Click-through-rate (CTR) is a common metric for these models. By considering the user's next job (position or company) as a recommendation item, and the ground truth job transition as the interaction ("click"), the career prediction is easily framed as a recommendation problem. In this paper, multiple established recommender architectures are used as backbones to test the performance of LLM knowledge integration.

In our dataset, individual careers are ordered job sequences. Recommendation architectures such as DIN and DIEN use attention structures to obtain information about each history item in the past behavior sequence [3, 4]. We also use multiple features in our prediction, and models such as Deep & Cross Network (v1 and v2) and DeepFM provide solid foundations for automatic feature engineering [5, 6, 7]. This paper uses these models as backbone models for testing the effectiveness of integrating LLM world knowledge features.

2.2 LLM Integration in Traditional Recommender Models

Previous studies have found that while LLMs have promising zero-shot ranking ability, they suffer from severe issues such as hallucinations, history length sensitivity, and candidate position bias [8, 9]. Therefore, there has been interest in integrating LLMs as part of traditional recommender models, rather than directly using LLM outputs for ranking. Models such as SeCor and KAR use vector representations of LLM output as information for later stages of the model [2, 1]. In this paper, we are interested in implementing and extending the KAR model. The specific implementation and improvements are specified in the following methodology section.

3 Methodology

3.1 Problem Statement

A user's career history is defined as an array of positions ordered by start date, where each position contains a role and a company. A user also has education information, which is defined as an array of education records, each containing a university, degree type, and field. The goal of the model is to use both the education information and previous positions in the position array to predict the next role / company for the user. In our experiments we examine two prediction problems: next role prediction and next company prediction.

3.2 Overview

The general model structure and philosophy of C-KAR remains close to the original KAR model [1]. The framework consists of three stages:

Knowledge Generation uses generated prompts to obtain relevant knowledge from LLMs. The prompt generator function constructs two types of prompts: reasoning for user information, and factual for role / company knowledge. The LLM world knowledge is then obtained in textual form.

Knowledge Adaptation converts the textual LLM world knowledge into fixed-size vector representations that can be used as features. First, the textual information is encoded using a frozen pre-trained language encoder. Then, the output embeddings are transformed using the hybrid-expert adaptor (HEA), which adapts and constructs features that are used in the knowledge utilization stage.

Knowledge Utilization ingests the LLM world knowledge as features to an existing backbone recommendation model, and leverages it to provide the final recommendation / prediction.

As with the original KAR framework, the hybrid-expert adaptor (HEA) and recommender model are trained jointly [1]. The pre-trained LLM and encoder language models are both frozen and do not undergo additional training.

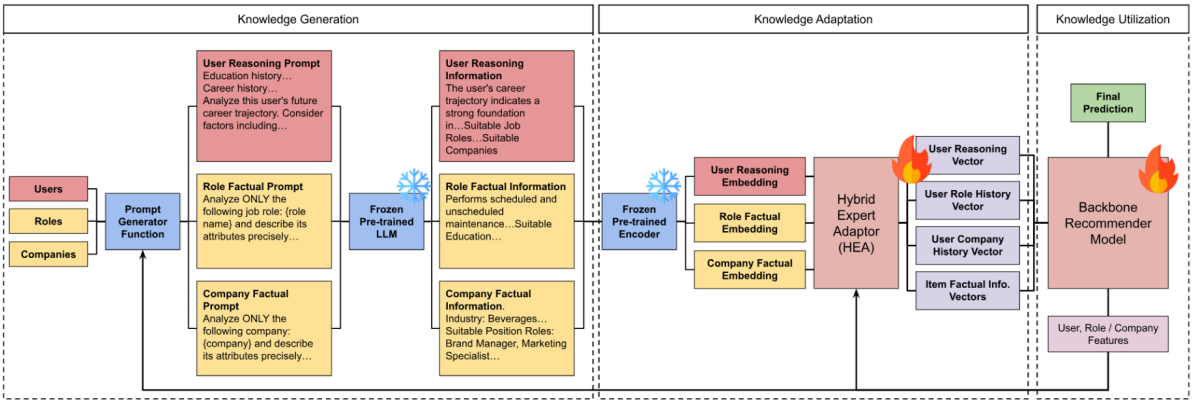


Figure 2: Overall framework of C-KAR model. The general framework is close to the original KAR model.

3.2.1 Knowledge Generation

In following with the original KAR paper, we design a prompt generator function, which creates two types of prompt: user reasoning prompt, and item factual prompt. These prompts induce the LLM to produce reasoning knowledge and factual knowledge respectively.

User reasoning prompt has the goal of obtaining reasoning knowledge about users with different career and education histories. The education histories (university, field, degree type) and career histories (position, company, tenure) are included in the prompt as textual information. The prompt also guides the LLM to output analysis regarding the user's career trajectory, rather than trying to directly predict the next job. The prompt's design intent is similar to the "preference reasoning

prompt" in the original KAR paper, and successfully leverages complex analysis abilities of LLMs to extract reasoning knowledge about each user [1].

System Prompt	You are an expert career intelligence analyst. Do not reveal chain of thought. Only provide the requested analysis.
Example Prompt	Education history: Associate in Business at Columbus State Community College… Career history: heavy equipment at Ohio National Guard Association in columbus metropolitan area, Ohio for 1.50 years; safety engineer at… Analyze this user's future career trajectory. Consider factors including career progression, education and career history, suitable job role, suitable company. Provide a concise explanation grounded in the supplied history.
Example Result (truncated)	The user's career trajectory indicates a strong foundation in both operational leadership and business administration… Suitable Job Roles:Given this background, suitable job roles would leverage their leadership experience and business understanding, such as: Operations Manager, Logistics Manager…Suitable Companies:Companies that benefit from structured operations, safety protocols, and supply chain expertise would be ideal. This includes: Manufacturing companies…

Table 1: Example prompt and output for user reasoning prompt.

Item factual prompt has the goal of obtaining general factual knowledge about roles and companies. Similarly to the KAR paper, this prompt seeks to obtain world knowledge about items that can compensate for the limited scope of our dataset [1]. In our case, this has significant implications for the handling of out-of-distribution (OOD) data, as even completely unseen items (e.g. companies that appear in inference but do not appear in the train dataset) can have meaningful associated information. This effect is apparent especially in OOD next-company prediction, recorded in our experiment section further below.

	Role	Company
System Prompt	You are a precise career intelligence expert. Do not output reasoning or chain of thought.	You are a precise career intelligence expert. Do not output reasoning or chain of thought.
Prompt	Analyze ONLY the following job role: {role name} and describe its attributes precisely (including but not limited to position role, suitable education, suitable career type). Return structured, factual attributes.	Analyze ONLY the following company: {company name} and describe its attributes precisely (including but not limited to suitable position role, suitable education, suitable career type). Return structured, factual attributes.
Example Result (truncated)	**Position Role:** * Performs scheduled and unscheduled maintenance, inspection… **Suitable Education:** * Completion of an accredited Aviation Maintenance Technician (AMT) program …	**Company:** The Coca-Cola Co. **Industry:** Beverages **Market Position:** Global leader in non-alcoholic beverages. … **Suitable Position Roles:** * Brand Manager * Supply Chain Analyst …

Table 2: Example prompts and outputs for role and company item factual outputs.

3.2.2 Knowledge Adaptation

The reasoning and factual knowledge produced by the knowledge generation stage are both direct outputs of LLM. However, text is unsuited to be used as features directly. Furthermore, even if we have access to the raw embeddings output by the LLM, the large, dense vector is not suited to the recommendation space. The original KAR paper recognizes this, and builds the knowledge adaptation pipeline with two sections: knowledge encoder and hybrid-expert adaptor (HEA) [1]. It is in the hybrid-expert adaptor that C-KAR implements significant improvements in architecture over the original KAR model.

Knowledge encoder

The knowledge encoder converts the textual reasoning knowledge and factual knowledge to dense vector representations. This can be achieved using a frozen pre-trained encoder model such as BERT [10].

Hybrid-expert adaptor (HEA)

The dense vector outputs from the knowledge encoder remain dimensionally large (e.g. 768 for BERT-Base, 1024 for Qwen3-Embedding-0.6B) and are not aligned with the recommendation space. To capture diverse information while adapting this reasoning and factual information, KAR proposes a Mixture of Experts inspired approach, which mixes shared and dedicated experts. Space transformation and dimensionality reduction is achieved by this architecture. We use this hybrid-expert adaptor as a base, with some key improvements:

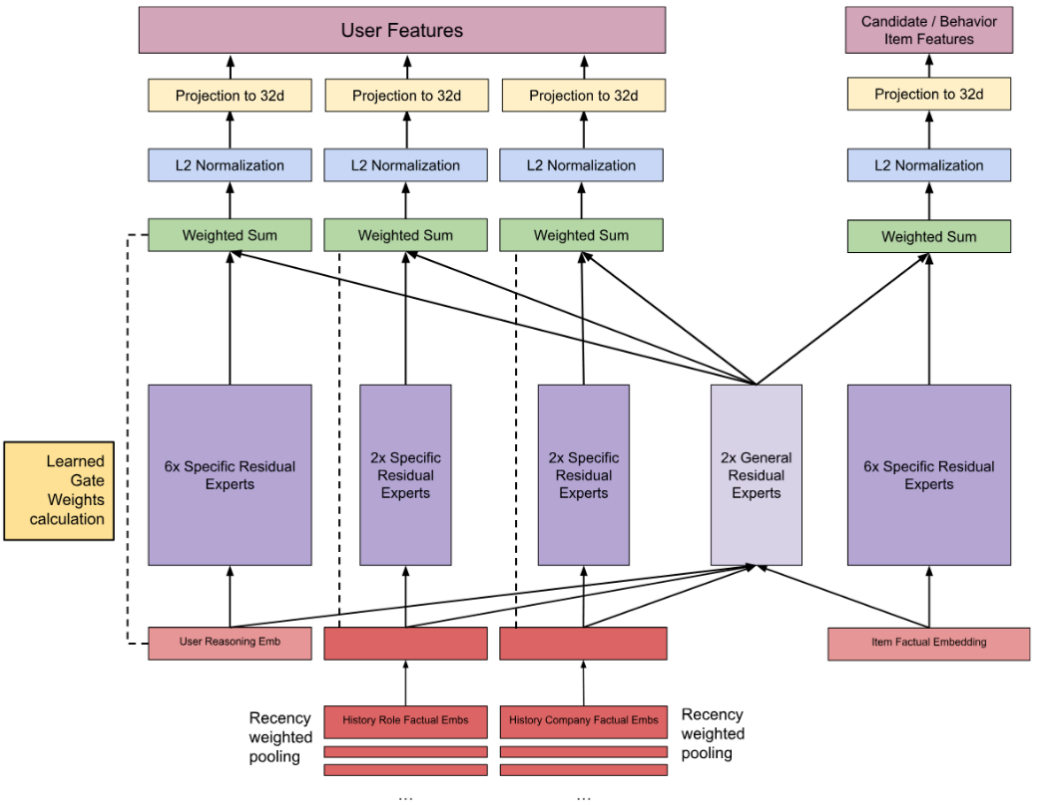


Figure 3: Overview of hybrid-expert adaptor of C-KAR.

1. **Residual HEA Experts:** In the hybrid-expert adaptor in the original KAR implementation, the experts are simple feed-forward MLPs [1]. It is found that by adding a ResNet-inspired skip layer, we obtain a better performance with significantly fewer parameters. The skip layer

allows information from the encoded knowledge vectors to be maintained rather than lost through deep networks.

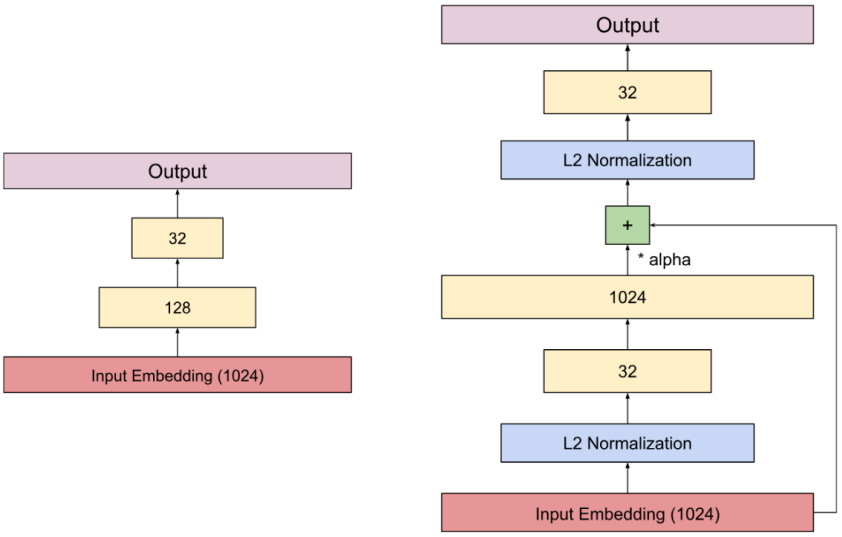


Figure 4: Hybrid-expert adaptor expert of original KAR (left) and C-KAR (right).

2. **Recency Pooled Histories as a User Feature:** We also find that simple pooling on user historical items (roles / companies) can serve as useful features. We perform pooling on the item factual embeddings in user history with exponential dropoff, and pass them as separate modalities of features to be adapted in the hybrid-expert adaptor. This creates two more useful features, user role history and user company history, for the knowledge utilization stage.

3.2.3 Knowledge Utilization

From the knowledge adaptation stage, we have obtained four different augmented knowledge vectors: 1. User reasoning augmented vector, 2. Item (company / role) factual augmented vector, 3. User role history augmented vector, 4. User company history augmented vector.

In the original KAR model, a simplistic approach is used where the augmented vectors are treated as additional input features to a traditional recommender model [1]. C-KAR follows the same philosophy as KAR, and is independent of the design and loss function of backbone model, which allows for use with a large variety of backbone recommender models.

The significant improvement of C-KAR over KAR in the knowledge utilization stage is the use of **item factual augmented vector in individual user behavior items**. The original KAR implementation only uses item factual knowledge for the candidate item that is being scored. Adding factual information for every item in past user behavior provides more context to the model, especially on rare or out-of-distribution (OOD) items that may lack enough training data to build a strong embedding representation.

4 Experiment

In order to gauge the performance of C-KAR on practical career-prediction tasks, we perform experiments using multiple backbone models in this section. We further perform ablation tests to gauge the effectiveness of each of our three improvements upon KAR.

4.1 Setup

4.1.1 Dataset

The dataset used is based on sampled users from the US LinkedIn dataset, limited to only those with target positions with start date > year 2021 (post-COVID period). We ensure that all input career history sequences have length >= 3.We also filter the companies to only the top 8000 companies with the most positions, and remove all users with any companies in their career history not within them.

From this dataset, we construct the train set and two test sets. We take the subset with targets with start date > year 2023 to be the test set. Then, we randomly sample 600 companies to be Out-of-Distribution (OOD), and remove all users with any of those companies in their career histories from the train or test sets. We extract these users to a new OOD test set, which is used to gauge the performance of the model on unseen data.

In total, we have: 31,504 datapoints in the train dataset, 8,420 datapoints in the test dataset, 3,249 datapoints in the OOD dataset. Each datapoint contains the career history, education history, and the target role and company. There are 1500 unique roles and 8000 unique companies (including OOD) in our dataset.

Furthermore, the backbone models being tested require a retrieved set of candidate items in practice. We simulate a perfect retrieval system, where the ground truth item is always in the retrieved candidate set. We construct this for each datapoint by selecting the 49 other semantically closest items to the ground truth item (calculated using the encoded vector representation of the item factual output from the LLM), resulting in a set of the top-50 candidates closest to the ground truth.

4.1.2 Backbone Models

By considering the next career position (role or company) as the next interacted with item, the career prediction task is easily framed as a recommendation problem. Therefore, we focus on using click-through-rate (CTR) models. As in the KAR paper, we select both User Interaction Models and Feature Interaction Models [1]. User Interaction Models model user past behavior sequences, and include DIN and DIEN models [3, 4]. Feature Interaction Models focus on the interactions between feature fields, and include DCNv1, DCNv2, and DeepFM models [5, 6, 7].

4.2 Metrics

This subsection details the metrics used to evaluate performance of the different models.

AUC (Area Under Curve) is first calculated separately within each 50-candidate set. The positive candidate's raw logit is compared with every negative logit. The AUC is this value averaged across all rows.

$$AUC_i = \frac{\#(z_{positive} > z_{positive}) + 0.5 * \#(z_{positive} = z_{negative})}{59}$$

LogLoss, as referenced in the results, is a 50-candidate softmax cross-entropy loss, across all 50 ranked candidates. The loss as reported during evaluation is averaged across all rows. The same loss is used during training.

$$p_y = \frac{exp(z_y)}{\sum_{j=1}^{50} exp(z_j)}$$

$$LogLoss = \frac{1}{N}\sum_{i=1}^{N} - log(p_{i,y_i})$$

Recall@K represents how often the ground truth label appears in the top K results.

4.3 Results

The models are trained with different backbone models on the train dataset, and evaluated on both the regular test and OOD test sets. Training and evaluation is done separately for role and company prediction tasks (i.e. separate models are trained and evaluated for each task).

The models are trained with LogLoss (as described in the metrics section above) as the loss function. Training ends at 10 epochs, with 3 epochs early stopping implemented.

The legend "Base" denotes the base recommender model without any world knowledge integration. "KAR" denotes the model with knowledge augmentation implemented as in the KAR model. "Ours" denotes our C-KAR, which implements our improved version of knowledge augmentation.

	AUC			**LogLoss**			**Recall@1**			**Recall@3**			**Recall@5**		
Backbone	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**
DIN	0.9021	0.9062	**0.9140**	2.2327	2.1868	**2.1424**	0.4331	0.4327	**0.4403**	0.6008	0.6083	**0.6201**	0.6975	0.7029	**0.7175**
DIEN	0.8994	0.9012	**0.9101**	2.3382	2.3163	**2.2537**	0.3658	0.3654	**0.3723**	0.5773	0.5790	**0.5950**	0.6880	0.6894	**0.7034**
DCNv1	0.8958	0.9009	**0.9130**	2.3743	2.3316	**2.2381**	0.3533	0.3711	**0.3791**	0.5589	0.5765	**0.6082**	0.6755	0.6840	**0.7204**
DCNv2	0.9016	0.9098	**0.9180**	2.3187	2.2497	**2.1804**	0.3654	0.3810	**0.3957**	0.5888	0.6013	**0.6238**	0.6947	0.7109	**0.7356**
DeepFM	0.8946	0.9038	**0.9094**	2.3883	2.3110	**2.2775**	0.3530	**0.3758**	0.3673	0.5590	0.5878	**0.5930**	0.6695	0.6982	**0.7075**

Table 3: AUC, LogLoss and Recall@K for **Roles** (IID) of different backbone models and knowledge adaptors.

	AUC			**LogLoss**			**Recall@1**			**Recall@3**			**Recall@5**		
Backbone	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**
DIN	0.9170	0.9105	**0.9261**	1.9480	2.0193	**1.7788**	0.5533	0.5425	**0.5849**	0.6941	0.6793	**0.7148**	0.7626	0.7462	**0.7793**
DIEN	0.9075	0.9208	**0.9271**	2.1578	1.8814	**1.7151**	0.4924	0.5563	**0.6004**	0.6471	0.6854	**0.7208**	0.7278	0.7552	**0.7819**
DCNv1	0.9018	0.9047	**0.9064**	2.2111	2.2363	**2.1814**	**0.4670**	0.4517	0.4435	0.6190	0.6188	**0.6213**	0.7031	0.7081	**0.7112**
DCNv2	0.9059	0.9068	**0.9194**	2.1703	2.1206	**1.9827**	0.5055	0.5110	**0.5271**	0.6498	0.6586	**0.6797**	0.7228	0.7309	**0.7526**
DeepFM	0.8951	0.9016	**0.9126**	2.4184	2.3111	**2.1129**	0.4468	0.4365	**0.4660**	0.6093	0.6017	**0.6417**	0.6952	0.6931	**0.7286**

Table 4: AUC, LogLoss and Recall@K for **Companies** (IID) of different backbone models and knowledge adaptors.

	AUC			LogLoss			Recall@1			Recall@3			Recall@5		
Backbone	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**
DIN	0.9074	0.9096	**0.9179**	2.1583	2.1441	**2.0863**	0.4444	0.4380	**0.4512**	0.6143	0.6150	**0.6433**	0.7048	0.7125	**0.7362**
DIEN	0.9051	0.9054	**0.9132**	2.2881	2.2788	**2.2139**	**0.3764**	0.3681	0.3758	0.5965	0.5928	**0.6147**	0.6993	0.7024	**0.7138**
DCNv1	0.8989	0.9052	**0.9152**	2.3351	2.2785	**2.2060**	0.3740	**0.3780**	0.3727	0.5737	0.5913	**0.6165**	0.6768	0.6916	**0.7230**
DCNv2	0.9060	0.9120	**0.9204**	2.2716	2.2169	**2.1466**	0.3807	0.3853	**0.4014**	0.5956	0.6125	**0.6310**	0.6996	0.7187	**0.7402**
DeepFM	0.8976	0.9054	**0.9120**	2.3494	2.2789	**2.2542**	0.3620	**0.3777**	0.3613	0.5734	0.5916	**0.6045**	0.6821	0.7027	**0.7110**

Table 5: AUC, LogLoss and Recall@K for **Roles** (OOD) of different backbone models and knowledge adaptors.

	AUC			LogLoss			Recall@1			Recall@3			Recall@5		
Backbone	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**	**Base**	**KAR**	**Ours**
DIN	0.7290	0.7403	**0.7937**	4.0850	3.7338	**3.2857**	0.2859	0.2644	**0.3327**	0.4131	0.4032	**0.4789**	0.4838	0.4749	**0.5565**
DIEN	0.6455	0.7496	**0.8006**	5.1444	3.7272	**3.1883**	0.2216	0.2899	**0.3780**	0.3373	0.4174	**0.5091**	0.4007	0.4977	**0.5759**
DCNv1	0.6522	0.6755	**0.6980**	4.8244	4.7919	**4.2852**	0.1708	0.1801	**0.1976**	0.2915	0.3127	**0.3410**	0.3789	0.3835	**0.4232**
DCNv2	0.6741	0.6918	**0.7451**	4.8628	4.3233	**4.1684**	0.2225	0.2333	**0.2619**	0.3447	0.3367	**0.4192**	0.4201	0.3998	**0.4866**
DeepFM	0.6458	0.6848	**0.7322**	5.3771	4.5256	**4.1736**	0.1631	0.1511	**0.2118**	0.2798	0.2822	**0.3669**	0.3496	0.3638	**0.4561**

Table 6: AUC, LogLoss and Recall@K for **Companies** (OOD) of different backbone models and knowledge adaptors.

The experiment results show that our C-KAR model brings consistent improvements in AUC and LogLoss compared to base recommender and KAR recommender models. Of note are the significant improvements on company prediction for the OOD dataset (Table 6), exemplifying the importance of LLM world knowledge for handling OOD data.

4.4 Ablation Testing

We perform ablation testing in order to investigate the effects of each of our improvements to the original KAR model. The legend “Ours” denotes our C-KAR model, while “Without Residuals”, “Without History Pooled Emb.”, and “Without Behavior Item Factual Knowledge” denote a C-KAR variant that removes the specific improvement, and replaces it with its implementation in the KAR model where applicable.

The models are trained with LogLoss (as described in the metrics section above) as the loss function. Training ends at 10 epochs, with 3 epochs early stopping implemented.

	AUC	LogLoss	Recall@1	Recall@3	Recall@5
Ours	**0.9140**	**2.1424**	0.4403	0.6201	0.7175

Without Residuals	0.9117	2.1524	0.4341	0.6165	0.7146
Without History Pooled Emb.	0.9121	2.1497	0.4413	0.6222	**0.7188**
Without Behavior Item Factual Klg	0.9116	2.1561	**0.4418**	**0.6238**	0.7172

Table 7: AUC and Recall@K for **Roles** (IID) of different ablation tests.

	AUC	**LogLoss**	**Recall@1**	**Recall@3**	**Recall@5**
Ours	**0.9261**	**1.7788**	**0.5849**	**0.7148**	**0.7793**
Without Residuals	0.9197	1.8969	0.5550	0.6956	0.7672
Without History Pooled Emb.	0.9232	1.8280	0.5697	0.7064	0.7711
Without Behavior Item Factual Klg	0.9218	1.9284	0.5681	0.7001	0.7666

Table 8: AUC and Recall@K for **Companies** (IID) of different ablation tests.

	AUC	**LogLoss**	**Recall@1**	**Recall@3**	**Recall@5**
Ours	**0.9179**	2.0863	0.4512	**0.6433**	**0.7362**
Without Residuals	0.9161	**2.0801**	0.4524	0.6260	0.7211
Without History Pooled Emb.	0.9152	2.0823	0.4488	0.6337	0.7261
Without Behavior Item Factual Klg	0.9111	2.1279	**0.4534**	0.6140	0.7156

Table 9: AUC and Recall@K for **Roles** (OOD) of different ablation tests.

	AUC	**LogLoss**	**Recall@1**	**Recall@3**	**Recall@5**
Ours	**0.7937**	**3.2857**	**0.3327**	**0.4789**	**0.5565**
Without Residuals	0.7680	3.5882	0.2804	0.4226	0.5057
Without History Pooled Emb.	0.7858	3.3615	0.3170	0.4598	0.5392
Without Behavior Item Factual Klg	0.7851	3.5790	0.3035	0.4666	0.5399

Table 10: AUC and Recall@K for **Companies** (OOD) of different ablation tests.

The results show that each of the three improvements tested provide quantifiable improvements to AUC and LogLoss for the model.

5 Future Works

It is possible that retrieval-augmented generation (RAG) techniques can be used to augment the prompt generator function, in order to retrieve even more relevant and focused knowledge from the LLM. A related experiment was conducted in the process of writing this paper, by retrieving the top-K most similar users via conventional collaborative filtering (CF) methods, and providing it as part of the reasoning prompt to the LLM. However, the experiment yielded inconclusive to negative results, potentially due to the LLM output trying to rank candidate items directly, rather than providing LLM reasoning knowledge. This may have overlapped with the recommendation ability the backbone

model was supposed to learn, and harmed final training performance. In future works, it may be worth exploring better prompt formulations to avoid this scenario, and potentially create more worthwhile LLM knowledge-based features.

6 Conclusion

Our work proves that the application of LLM world knowledge in recommender models is highly beneficial to career sequence prediction applications. The world knowledge can be applied as additional features to traditional recommender models and produce better results. From KAR, we design C-KAR, which brings multiple performance improvements. Compared to KAR, C-KAR uses residuals in experts, provides recency-pooled user item history information as additional features, and incorporates item factual knowledge as features of individual items. We prove that C-KAR shows superior performance on both next-role-prediction and next-company-prediction tasks, compared to both KAR and traditional unaugmented recommender models.

References

[1] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24)*. Association for Computing Machinery, New York, NY, USA, 12–22. https://doi.org/10.1145/3640457.3688104

[2] Shirui Wang, Bohan Xie, Ling Ding, Xiaoying Gao, Jianting Chen, and Yang Xiang. 2024. SeCor: Aligning Semantic and Collaborative Representations by Large Language Models for Next-Point-of-Interest Recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24)*. Association for Computing Machinery, New York, NY, USA, 1–11. https://doi.org/10.1145/3640457.3688124

[3] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Kuang Yu, Yang Li, Yanqi Zheng, Hongbo Li, Kun Gai, and Jian Xu. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*. ACM, New York, NY, USA, 1059–1068. doi.org

[4] Guorui Zhou, Chenru Song, Ying Fan Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep Interest Evolution Network for Click-Through Rate Prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, No. 01, 5941-5948. DOI: 10.1609/aaai.v33i01.33015941

[5] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the ADKDD'17 (ADKDD'17)*. Association for Computing Machinery, New York, NY, USA, Article 12, 1–7. https://doi.org/10.1145/3124749.3124754

[6] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference 2021 (WWW*

'21). Association for Computing Machinery, New York, NY, USA, 1785–1797. https://doi.org/10.1145/3442381.3450078

[7] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. In Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI'17). AAAI Press, 1725–1731.

[8] Chumeng Jiang, Jiayin Wang, Weizhi Ma, Charles L. A. Clarke, Shuai Wang, Chuhan Wu, and Min Zhang. 2025. Beyond Utility: Evaluating LLM as Recommender. In *Proceedings of the ACM on Web Conference 2025 (WWW '25)*. Association for Computing Machinery, New York, NY, USA, 3850–3862. https://doi.org/10.1145/3696410.3714759

[9] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large Language Models are Zero-Shot Rankers for Recommender Systems. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR* 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part II. Springer-Verlag, Berlin, Heidelberg, 364–381. https://doi.org/10.1007/978-3-031-56060-6_24

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1 (Long and Short Papers), pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.